AJ Pilot

Autopilot for your servers. Deploy apps, manage servers and connect domains from one self-hosted panel
.

> Formerly "Auto Deploy". Technical names on your servers keep the old spelling so existing apps stay managed: the `auto_deploy` database, `/opt/auto-deploy/…` folders and `auto-deploy.*` Docker labels.

Phase 1 (built): server and database management. Add an Ubuntu server with its SSH credentials, connect, and read a complete system profile — hardware, storage, network, listening ports, processes, services, updates, security posture, installed tooling, Docker and MySQL state. Connect to the MySQL databases running on those servers and browse them down to columns and indexes. Git, Docker Hub and Cloudflare credentials are stored and verified so the deployment phase can use them.

Phase 2 (built): services and CI runners. Manage every systemd service on a server — start, stop, restart, enable, read its journal, and create new units from a form. Install a self-hosted CI runner on any of your servers straight from a git account, and see the runners each repository has.

Phase 3 (built): installations. A catalog of things to install — Docker, Docker Compose, MySQL, PostgreSQL, MongoDB, Redis, EMQX and Elasticsearch. Everything except Docker itself runs as a container, on a port you choose and can change later.

Phase 4 (built): custom services. Point the panel at a Node.js repository — a service, Next.js, NestJS, Angular, React — and it checks the project, builds it into a Docker image on one of your servers and runs it there, as one container or several on consecutive ports, optionally pushing the image to Docker Hub first.

Phase 5 (built): organisations and people. The panel signs you in. The first account ever created is the super admin; after that an admin adds people to their organisation with a role — add, edit, delete, or view only. Servers, credentials, runners and installations all belong to an organisation and are invisible to every other one.

Quick start

The panel stores everything in **MySQL**. Point it at a database you control:

```
cp .env.example .env     # then fill in DB_HOST / DB_USER / DB_PASSWORD
npm install
npm start                # http://localhost:4000
```

The `auto_deploy` database and all its tables are created automatically on first boot — you only need a MySQL user that may `CREATE DATABASE`, or an empty database that already exists.

No MySQL to hand? A throwaway one:

```
docker run -d --name aj-pilot-db -e MYSQL_ROOT_PASSWORD=secret -p 3306:3306 mysql:8
```

`npm run dev` restarts on file changes. If `APP_ENCRYPTION_KEY` is unset, a random `data/master.key` is generated on first boot.

Coming from the SQLite version

If `data/auto-deploy.db` is still present, its servers, system profiles, credentials and activity log are copied into MySQL automatically the first time you boot against an empty database, and the file is renamed to `.imported`. Secrets move across as ciphertext and keep working, because the master key does not change. The import only ever runs into an empty database, so it cannot overwrite live data or run twice.

The front page

Anyone who opens the panel without a session gets a landing page rather than a bare login box: what the tool does, how it works, what it does with your secrets, and who it is for — with **Sign in** in the header and at the foot of the page. The sign-in card opens over it, and closes again with Escape or a click outside. On a panel that has never been set up, those buttons say **Set up your panel** instead and open the first-run form.

Signed in, the landing page is replaced by the panel itself.

Signing in

The first time the panel is opened it has no accounts, so it asks for one: your name, email, a password and the name of your **organisation**. That first account is the **super admin** — the only role that can create organisations and hand out the super admin role itself. Everyone after that is added by an admin from **Settings** → **Team**; there is no public sign-up.

The header

Once you are in, every page carries a header: the page you are on and your organisation on the left, and on the right the organisation picker (super admins only) and your own account — avatar, name and role. That button opens a menu with **Edit profile**, **Change password**, **Settings** and **Sign out**.

Settings

Settings gathers everything that is not a server:

Tab	What is there
Your account	your name, email, role and what that role may do, with edit-profile and change-password
Team	the people in this organisation and their roles — only for those who may manage people
Organisations	every organisation with its member, server and credential counts; a super admin creates, renames, switches and deletes them here
About this panel	what this organisation holds, the database in use and how long the panel has been up

Organisations

An organisation owns its servers, credentials, git accounts, runners and installations. Nothing is shared between organisations, and nothing is owned by a person — remove somebody and their organisation's servers stay exactly where they were.

A super admin can work in any organisation and switches with the picker in the header. Everybody else only ever sees their own.

Roles

	View	Add	Edit	Delete	Manage people	Organisations
Super admin	✓	✓	✓	✓	✓	✓
Admin	✓	✓	✓	✓	✓	—
Editor	✓	✓	✓	—	—	—
View only	✓	—	—	—	—	—

Roles are enforced by the API, not by the buttons: every `POST` needs *add*, every `PUT` needs *edit* and every `DELETE` needs *delete*, whatever the browser sends. The read-only actions — testing a connection, collecting a system profile, verifying a credential, reading a database or refreshing a runner's status — stay open to **view only**, so someone can look without being able to change anything. The UI then hides what a role cannot use, so nobody is offered a button that will only refuse them.

An admin can add people and change their roles, but cannot promote anyone to super admin, cannot create organisations, and cannot touch anyone outside their own organisation. Changing a role, disabling an account or changing a password ends that person's sessions immediately. The only super admin cannot be deleted, demoted or disabled — make somebody else one first.

Sessions

Signing in sets an `HttpOnly`, `SameSite=Lax` cookie holding a random token; only its SHA-256 is stored, so the session table is useless to anyone who reads it. Passwords are hashed with scrypt. Set `AUTH_SECURE_COOKIE=true` when the panel is behind HTTPS, and `AUTH_SESSION_DAYS` to change how long a sign-in lasts (14 days by default).

Upgrading a panel that had no accounts

Nothing is lost. On the first boot with this version, every existing server, credential, runner and installation is moved into one organisation automatically, and the setup screen then adopts that organisation under whatever name you type. The console says so when it happens:

```
[db] Moved existing servers and credentials into "Default organisation" (added org_id to servers, …).
```

Adding your first server

1. Open http://localhost:4000 and click **+ Add server**.
2. Fill in host/IP, SSH port, username, and either a password or a private key.

3. Click **Test connection** to confirm the credentials before saving.
4. **Save & connect** stores the server and immediately collects its full system profile.

The × in the popup's top corner, or the **Esc** key, closes it at any step — choosing how to start, the setup guide, or the form — without saving anything.

Use **Fetch system details** any time to re-read a server. Each collection is stored as a snapshot in `server_facts`, so you keep a history of how a machine changed.

Setting up a new machine with one file

Turning an ordinary PC, laptop or mini-PC into a server? **+ Add server** → **Start from a blank machine** walks through installing Ubuntu, then gives you **one setup file** that does the rest. Tick what you want, press **Download aj-pilot-setup.sh**, copy the file onto the machine and run:

```
sudo bash aj-pilot-setup.sh
```

Option (ticked by default unless noted)	What the file does
SSH	installs and starts the SSH server on the port you choose (22 unless you change it) — always on
Never sleep	the machine never sleeps, suspends or hibernates, even if something asks it to
Lid closed keeps running	on a laptop, closing the lid does nothing
Screen turns off	the screen goes dark after the minutes you choose; the computer keeps running
Wi-Fi never dozes	turns Wi-Fi power saving off, and keeps it off after a restart
Wake-on-LAN	lets another device on the network switch it on
Time zone and clock sync	your time zone (taken from your browser) and automatic time
Firewall	allows SSH (and web ports 80/443, if ticked), then turns the firewall on
Block password guessing	fail2ban bans addresses that keep failing to sign in over SSH
Security updates	installed automatically
Passwordless sudo for your user	the panel can manage the machine without you typing a sudo password
Swap	adds a swap file on machines with under 4 GB of memory and none set up
Docker *(off by default)*	installs Docker now — the panel can also do it later
Server name / SSH public key *(optional)*	renames the machine; lets a key sign in as well as your password

The file is safe to run again — it changes only what is not set up yet. It works out what kind of machine it is on: in a virtual machine or a cloud server it leaves out the laptop, screen, Wi-Fi and power settings; on a machine without a battery it leaves out the lid setting. Each step is checked on its own, so one that fails does not stop the others; an SSH or sudo change that would not pass its own check is put back as it was. It waits for Ubuntu's own updates to finish rather than failing, says plainly when there is no internet, and refuses anything that is not Ubuntu or Debian. Run it with `--dry-run` to see what it would do without changing anything, and `--help` for every option.

At the end it prints a summary of every step and **exactly what to type into AJ Pilot**: the address, the SSH port and the username. Press **Next** in the guide (or use **My server is ready** in **+ Add server**) and paste that box into **Paste what the setup file printed** — the address, port and username fill in by themselves. You only need your login password, and no sudo password when passwordless sudo was ticked. Everything it did is also written to `/var/log/aj-pilot-setup.log` on the machine.

One thing no file can do: **switching back on after a power cut** is a BIOS setting (*Restore on AC power loss → Power On*). The guide shows where to find it.

Editing a server

Edit on a server's card — or **Edit server** on its own page — reopens that form with everything filled in: name, host, port, SSH user, authentication, tags and notes. The secret boxes come up empty and say *unchanged*; typing in one replaces what is stored, leaving it empty keeps it. **Test connection** works here too, so a moved host or a rotated password can be checked before it is saved. Renaming to a name another server already has is refused by name rather than as a database error, and switching the authentication kind without supplying the password or key it needs is refused as well. Editing needs the **edit** permission.

Servers reached through your PC (connectors)

When only your own PC can reach a server — its VPN is connected there, like running the project locally — run a **connector** on it. **Servers → Connectors → Create connector** gives you a ready-to-run download with this panel's address and a private token built in. Run it on that PC (Node.js 22+; on Windows double-click the `.cmd`) and connect the VPN. There is nothing to set per server: the panel tries every server directly first, and when it cannot reach one it goes through a running connector instead (remembering which way worked for ten minutes). SSH, deploys, the file manager and the monitor all work this way; with the connector stopped those servers show offline.

The connector only makes an outgoing WebSocket connection to `/api/connectors/ws`, so nothing is opened on the PC (Cloudflare passes WebSockets through). `--allow 192.168.0.0/24` limits which addresses it will connect to. A new token disconnects the running copy.

The server page

The sidebar is fixed, so it stays put however far you scroll, and collapses to an icon rail with the « button next to the logo — the choice is remembered in the browser. Until you choose, a screen 1100px wide or less (a tablet or small laptop) starts with the icon rail, so pages keep most of the width. On a phone (720px or less) the sidebar is a menu that slides in from the left: tap ☰ in the top bar to open it, and it closes when you pick a page, tap outside it or press Esc.

Everything about one server sits behind a vertical tab rail on the left rather than one long scroll. The rail stays put as you scroll a long panel, and folds into a scrolling row of tabs under 900px wide:

Tab	Shows
Live	what the machine is doing right now, refreshed every few seconds — The live view
Overview	what is on this server, counted, then the headline tiles, host identity and CPU, memory and swap
Storage	filesystems, block devices, inode usage
Network	public IP, gateway and DNS, interfaces, every listening port

Tab	Shows
Processes	top processes by CPU and by memory, who is logged in
System	host MySQL, pending updates, security posture, installed tooling, nginx sites and cron
Apps	the custom services deployed onto this server, as full cards — Custom services
Docker	engine state, registry sign-in, networks, and the services installed here — Installations
Nginx	nginx and certbot, every domain it serves and whether each one is on SSL — Nginx and SSL
Cron	everything scheduled on the machine, with add, edit and delete — Scheduled jobs
Services	every systemd unit, live — Services
Runners	the CI runners installed on this server — CI runners

Overview, Storage, Network, Processes and System come from the stored profile and are instant. The rest talk to the server, so they are only fetched when you first open them — opening a server no longer starts several SSH sessions at once — and each carries a count once loaded (live CPU, apps, running services, running containers, runners), with a red marker when something has failed. Creating a service, installing something or adding a runner drops you on the tab that now shows it.

On this server

The **Overview** tab opens with a row of counts — everything on that machine, in one line each:

Tile	Says
IPv4	the address the machine answers on, and its public address
Containers	how many are **running**, and of how many, with stopped, restarting, paused and dead broken out beside it
Custom services	how many of the repositories you deployed here are running, and how many are deploying or broken
Installed services	the databases and the like from the catalog, running against total
Domains	how many nginx serves, how many are on SSL, whether nginx is up, and how long the nearest certificate has
Scheduled jobs	jobs in user crontabs, jobs from the system tables, and whether cron is actually running
System services	systemd units running, of how many exist, and how many have **failed**
CI runners	runner services up, of how many are on the machine, and how many are registered here
Docker storage	volumes, images and networks, with the compose version

Every tile opens the tab that owns it, so a number that looks wrong is one click from its detail.

The **IPv4** tile is the live one, not the one from the stored profile, and it shows two things only: the address the machine answers on, and its public address. "Answers on" means whichever address sits on the interface the default route leaves by — not simply the first `ip addr` prints, which is a different address on a host with a private interface and a public one. When the two are the same it says so rather than printing the address twice. Every other address the host holds is on the **Network** tab, which the tile opens. The public lookup has a two-second timeout and is allowed to come back with nothing: a server with no outbound access still gets its tile, and an answer that is not an IPv4 address — the HTML a captive portal returns — is discarded rather than shown.

The counts are one SSH pass, not one per subject — a summary that costs five connections is one nobody waits for. Two sources are folded together: the machine is asked what it is actually running, and the panel's

own tables are asked what it was told to run, because the gap between those is usually the interesting part. The stored profile below renders instantly as it always did; the counts fill in a moment later, and **Recount** refreshes them. Opening a server now starts that one connection, where before it started none.

The live view

Live is the machine as it is this second, not as it was when its profile was last collected. It holds **one** SSH session open and pushes a sample every five seconds over server-sent events — polling would mean an SSH handshake every five seconds per watcher. The session is dropped the moment you look at another tab or leave the page, and picked up again when you come back.

Each sample is taken on the server itself: `/proc` is read twice a second apart, so CPU busy, network throughput and disk I/O are real rates rather than guesses from single readings.

CPU	busy percentage with a five-minute sparkline, time waiting on disk, steal time on a VM, and a bar per core
Memory	used against total (from `MemAvailable`, not "free"), cache and buffers, swap use, with its own sparkline
Load	1, 5 and 15 minutes, per core, with how many processes are runnable
Throughput	network in and out per second, and disk read and write per second
Filesystems	every real mount with used, free and a bar
Containers	how many are running, restarting, paused, stopped or dead
Processes	the top five by CPU and the top five by memory

Containers are listed with their own live numbers — CPU, memory against limit, network in and out, from `docker stats` — beside their state, health, image and published ports. **Start**, **Stop**, **Restart** and **Logs** work on any container on the machine, not only the ones this panel installed. A row is held still while an action on it is in flight, so a button is never redrawn under your click.

When something is wrong

Every sample is checked against thresholds and what fails comes back as an alert, worst first, above everything else on the tab:

Alert	Warning	Critical
CPU	75%	90%
Memory	80%	92%
Any filesystem	80% full	90% full
Load per core	2	4
Swap in use	50%	—
CPU waiting on disk	25%	—
CPU stolen by the host	10%	—
Docker	a paused container	the daemon down, a container restarting in a loop, dead, or failing its health check

Each alert says what it means rather than just quoting a number — *"Only 3 GB left. A full disk stops databases and deployments dead."* The tab's counter turns red, and a **critical** alert also raises a toast the first time it appears, so a problem that starts while you are reading another part of the page still says so. An alert that clears can announce itself again if it comes back.

What gets collected

Group	Details
System	distribution, version, codename, kernel, architecture, machine ID, timezone, boot time, virtualization, init
CPU	model, vendor, vCPUs, sockets, cores/threads, clock, L3 cache, hypervisor
Memory	total / used / available / free, buffers, cached, swap
Storage	every filesystem with size, used, free and use%, inode usage, block devices with model
Network	interfaces (IPv4 + IPv6), default gateway, DNS servers, public IP, every listening port with its process and PID
Processes	total count, top 10 by CPU and by memory
Services	running systemd units and any failed units (the full list is read live — see Services)
Updates	dpkg/snap counts, pending updates, security updates, reboot-required flag
Security	ufw status, fail2ban, SSH port, PermitRootLogin, PasswordAuthentication, sudo users, active sessions
Tooling	docker, compose, git, node, npm, python3, nginx, certbot, psql, and more with versions
Docker	engine version, running/stopped counts, images, storage driver, full container list with ports
MySQL	installed version, service state, bind address, port, data directory and its size, whether it is localhost-only
Web / jobs	nginx sites-enabled, root crontab

The probe runs as a single remote script (src/lib/probe.sh) in one SSH round trip. Every section is best-effort — a missing tool yields an empty section rather than a failed run.

Privileges. Connecting as `root`, or as a user with passwordless sudo, gives the complete picture. Without either, process names on listening ports, firewall state and Docker details may be missing; the UI says so in a banner rather than silently showing gaps. If the SSH user needs a password for `sudo`, put it in the **Sudo password** field on the server form — managing services and installing runners need root.

Services

Every server page lists **every** systemd service on the host, read live over SSH rather than from the stored profile: running, stopped, failed and the ones that are only installed.

The tab opens with counters — **Running**, **Failed**, **Stopped**, **Starts on boot**, **Created here** and **All** — and clicking one shows just those services. When anything has failed, a red banner names it with **Restart** and **Logs** buttons beside each. A search box narrows the list by name or description.

Each service is a row: a coloured dot (green running, red failed, grey stopped), its name without the `.service` ending and its description, an **On boot** switch, and the actions. Failed services come first, then the ones created here. Services that keep the server reachable are marked 🛡.

Action	What happens
Click the name / Logs	Status, PID, memory, the unit file and the last 200 journal lines, with Start / Stop / Restart there too
Start / Stop / Restart	`systemctl <action>`, then the new state and journal tail come back
On boot switch	Enable / disable — whether it starts on boot
Delete	Only for units this panel created — stops, disables and removes the unit file

Stopping something that keeps the server reachable (`ssh`, `systemd-networkd`, `dbus`…) asks for confirmation first.

Adding a service

+ Add service writes a unit to `/etc/systemd/system`, runs `daemon-reload`, then enables and starts it. The form walks through four steps: **what to run** (name, description and the start command, with one-click templates for a Node script, an npm start script, a Python script or a shell script), **where and as whom** (working directory, user, group), **environment** (variables, an environment file or a saved environment) and **keep it running** — *Restart if it crashes*, *Always restart* or *Never restart*, how long to wait, and switches for starting on boot and starting now. The service type, what it starts after and replacing an existing unit are under **Advanced**.

The start command must be an absolute path — systemd does not run a shell, so `npm start` will not work whereas `/usr/bin/npm run start` will. The form warns as you type, and says so again before anything is written. If the unit fails to start, the error comes back with the journal attached instead of a bare exit code.

Units created here are recorded in `managed_services` and marked **created here** in the list, which is what makes them deletable from the UI. Nothing else on the server can be deleted by the panel.

Installations

The **Installations** page is a catalog of cards. Click **Install**, pick one of your servers, and the panel does the rest. The same popup is on every server's **Docker** tab as **+ Install a service** — opened from there it arrives with that server already chosen, and its **Service** picker installs anything in the catalog without leaving the page. Changing the picker redraws the form for that service: its versions, its default port and its own settings.

	Installed as	Default port
Docker Engine	on the host, from `get.docker.com`	—
Docker Compose	on the host, the compose v2 plugin	—
Nginx	on the host, from the Ubuntu packages, started and enabled at boot	80 + 443
MySQL	container — `mysql`, named volume at `/var/lib/mysql`	3306
MongoDB	container — `mongo`, named volume at `/data/db`	27017
Redis	container — `redis`, append-only persistence at `/data`	6379
EMQX	container — `emqx`, named volume at `/opt/emqx/data`	1883 + dashboard 18083
Elasticsearch	container — `elasticsearch`, named volume at `/usr/share/elasticsearch/data`	9200

Nothing but Docker and Nginx is installed onto the machine. Databases are never apt-installed; they run as containers with a named volume, so the data survives a rebuild and nothing is left behind on the host when you remove them.

Nginx goes onto the host because it fronts everything else: it is the web server your domains, sites and certificates live on. It does not need Docker. If Nginx is already there, installing again just makes sure it is running and starts at boot; if a firewall (ufw) is on, HTTP and HTTPS are opened in it. Its sites, certificates and logs are then managed on the server's **Nginx** tab.

A missing requirement is said first. Some services need another one on the server before they can be installed: every container service and Docker Compose need **Docker**, and the **Nginx Log Analyzer** needs **Nginx**. Picking a server checks it straight away, and if what is needed is missing you get a warning such as "*Nginx is not installed on this server*" with a button to install it there first — the **Install** button stays off until it is. This happens before anything is attempted, not after it fails, and the same check guards the API, so it cannot be worked around. Docker and Nginx themselves need nothing first.

Each service's form covers the version, the container name, the **port**, which Docker network to join, and whether it is reachable from anywhere or only from the server itself, plus its own settings — root password and an initial database for MySQL, root credentials for MongoDB, a password for Redis, a password for the `elastic` user and the JVM heap for Elasticsearch. Passwords are encrypted with the same master key as every other secret and are never returned by the API.

Services that answer on more than one port

A broker or a search engine does not live on a single port, so those get a box each. Every port box is checked against what the server already publishes, and "*in use by …*" appears beside any that clashes before you install rather than after it fails.

EMQX publishes MQTT on the first port and its **dashboard** on its own (18083 by default), with optional boxes for MQTT over WebSocket (8083) and over TLS (8883) — leave those empty and they are not published. The dashboard user is `admin` and the password you give it must be at least 8 characters, because EMQX refuses to keep its own default.

Elasticsearch runs as a single node with its indices on a named volume, and the optional transport port (9300) is only needed if more nodes will ever join it. A password sets up the `elastic` user and turns authentication on; leave it empty and the API is open to anyone who can reach the port. HTTPS is turned off so the API answers on plain `http://server:9200` rather than with a self-signed certificate nothing else here would trust. **JVM heap** defaults to `1g` — set it to about half the memory you can spare. Before the container starts, the panel sets `vm.max_map_count=262144` on the host and writes it to `/etc/sysctl.d/99-auto-deploy.conf`, because Elasticsearch will not start without it and no container can set it for itself.

Changing a port

Docker cannot re-publish a port on a running container, so **change** next to the port rebuilds the container. A service's other ports are kept as they were installed. The named volume is kept, so the data comes back with it — verified by a Redis instance moved from 6399 to 6398 that reloaded its append-only file on the other side. If the new container fails to start, the previous one is renamed back and restarted, so a bad port never costs you the service.

The Docker tab

A server's **Docker** tab opens with a status header — whether the engine is running, its version, Docker Compose and the registry it is signed in to — with **+ Install a service** and **Refresh** beside it, then four counters: running containers, stopped ones, volumes (and how many are used by nothing) and networks.

Containers come first, as cards. Each card has a coloured edge for its state (green running, yellow paused or restarting, red exited or dead) and shows:

- a mark for who put it there — 🚀 one of your apps, 📦 a service installed by the panel, 🐳 anything else;
- its name (click to open the container's own page), its state and how long it has been in it;
- the image it runs;
- its published ports as chips — `4200 → 80` — where a port published on all addresses is a link that opens `http://<server>:<port>`;
- **Logs**, **Details**, **Start** / **Stop** / **Resume** and **Restart**.

Every running container's card also shows **what it uses right now**: CPU as a share of the whole server, memory with a bar against its limit, network received and sent, disk read and written, and its process count. Above the cards, a **Docker usage** strip adds them up — CPU and memory used by all containers together, the busiest container — and shows **Docker's disk use**: images, containers, volumes and build cache, with how much of it could be reclaimed by cleaning up. The numbers load a moment after the page (they come from `docker stats` and `docker system df` on the server) and refresh every 15 seconds while the tab is on screen; **Pause** stops that.

A search box and **All / Running / Stopped / Apps / Installed** filters sit above them; containers that are restarting or dead are listed first. **Installed services**, **Volumes**, **Networks** and **Registry sign-in** follow as folding sections, each with its count in the header; one that is open stays open after a refresh.

The container page

Clicking a container's name or **Details** opens a page of its own — not a popup — so it has room, keeps its place when you refresh, and the browser's back button returns you to the server's Docker tab. Its header has the container's state and health, **Start** / **Stop** / **Pause** / **Resume** / **Restart** / **Remove**, and a **Download** menu. Below it are tabs:

Tab	What it shows
Overview	state and uptime, restarts, size, image, command, working folder, memory and CPU limits, labels
Live stats	CPU, memory against its limit, network and disk traffic per second with small charts, processes and restarts — updated every 3 seconds
Logs	**live**: the last 200 lines, then each new line as the container writes it — also after it is stopped and started again. **Follow** keeps the newest in view (scroll up to read and it lets go); **Pause**, **Clear**, **Timestamps**, a filter box and **Download .log**. Error lines show in red, warnings in amber
Volumes	everything mounted into the container: each named volume with its size, where it lives on the server, what else uses it, and **Download**; folders from the server and temporary mounts are listed too
Ports & networks	published ports and the Docker networks it is on, with addresses
Environment	the names of its variables (values are not shown here)

Tab	What it shows
Processes	what is running inside it

The live numbers and live logs stop as soon as you leave the page, and pick up again when you come back.

Downloads (needs a role that can edit):

- **Compose file (YAML)** — a `docker-compose.yml` that describes this container: image, command, environment variables *with their values*, ports, volumes, networks, restart policy, limits and health check. Run `docker compose up -d` next to it to create the same container again, here or on another machine.
- **Backup (.tar.gz)** — one archive for this container only: the compose file, Docker's full description of the container, a `README.txt` saying how to restore it, and a copy of every volume it uses. Tick **Pause while copying** for a database, so the copy is taken while nothing is writing. Folders mounted from the server itself are named in the archive's `README.txt` but not copied — download those from the file manager.
- **Volume** — on the Volumes tab, or in the server's Volumes section, any named volume downloads on its own as a `.tar.gz` of its contents.

These files can hold passwords and your data, so keep them somewhere safe. Every download is recorded in the activity log.

Registry sign-in

The Docker panel shows which registry accounts that server is signed in as — Docker Hub and anything else in its `config.json` — with the username and where the credentials are stored. If it is signed in to nothing, it says so: public images still pull, but private ones will not, and Docker Hub's anonymous rate limit applies.

Sign in to a registry runs `docker login` on the server, either with a Docker Hub credential already stored in the panel or with a username and token typed in. Docker then keeps those credentials on that host exactly as it would had you run the command yourself; the panel reads back only the username, never the password. **Sign out** runs `docker logout` for that registry.

Docker networks

A server page has a **Docker** panel: engine and compose versions, container counts, and the server's networks as chips with **+ Add network** and a × to remove one. Containers on the same user-defined network reach each other by container name, which is what a database and the app using it normally want. Docker's built-in `bridge`, `host` and `none` cannot be removed, and a network with containers still attached says so rather than failing obscurely.

Docker volumes

The same panel lists every **named volume** on the machine — where a container's data actually lives — with its size, its mount point on the host, and **what is holding it**: each container using it, with a green dot when that container is running. A volume the panel created for something it installed says so ("*data for mysql*"). Sizes come from `docker system df`; an older daemon that will not report them leaves the column empty rather than dropping the volume.

The fact worth having is the one neither `docker volume ls` nor `docker ps` gives on its own: a volume with **nothing** attached is flagged, because it is either last week's database or free disk space and only you know which. Every volume has **Download**, which saves its contents as a `.tar.gz` (needs a role that can edit). Those with nothing attached — and only those — also get a **Remove** button, behind a confirmation that says the data goes with it. A volume belonging to a service this panel installed is refused: remove the service instead, which offers to delete its volume with it. Docker itself refuses to delete a volume a container still references, and the panel names the container rather than passing the error through.

The same panel lists **installed services** on that server — status, image, port, network — with Logs, Start/Stop, Restart, change-port and Remove, plus any other containers already on the machine that the panel did not install.

Custom services

Apps → + Custom service deploys a Node.js repository from one of your connected git accounts onto one of your servers, as a container. Only servers that are **online** are offered: one that is offline, failing its checks or rebooting is left out of the list until it is back.

It checks the project first

Pick the git account, repository and branch, and the panel reads `package.json` through the provider's API — nothing is cloned yet. **No `package.json` at the root means the repository is refused**, with the reason said plainly, rather than a build that fails ten minutes later. A project with no `start` script is refused the same way, with a hint about what to add.

What it finds becomes the suggested settings, all of which you can overrule:

Read from the repository	Becomes
dependencies	the **project type** (below) and its usual port
`engines.node`	the Node version (22 by default)
`package-lock.json` / `yarn.lock` / `pnpm-lock.yaml`	`npm ci`, `yarn --frozen-lockfile` or `pnpm --frozen-lockfile`
`scripts.build`	the build command
`scripts.start`, or `main`	the start command
`angular.json`	where Angular builds to, including v17+'s `browser/` folder
a `Dockerfile` at the root	the option to use the repository's own build instead

Project types

The type it picks is a suggestion — the dropdown lets you override it, which matters when a repository could reasonably be either.

Type	Runs as	Default port
Next.js	its own server, `npm run start`	3000
NestJS	`node dist/main.js` after a build	3000

Type	Runs as	Default port
Nuxt	`node .output/server/index.mjs`	3000
Node.js service	Express, Fastify, Koa, or any start script	3000
Angular	built, then served by **nginx**	80
React · Vite · Vue	built, then served by **nginx**	80
Static site	whatever lands in the output folder, served by nginx	80

The split matters: a front-end has no server to run, so `npm start` would give you a development server. The static types build the project with Node and then copy only the result into an nginx image — smaller, and correct. They ship an nginx config that falls back to `index.html`, so client-side routes survive a reload.

One container or several

Containers on the form is a list: one row per container, each with the port it publishes. **+ Add container** adds a row on the next free port, **Remove** takes one away, and the port on any row can be typed over — they do not have to be consecutive, so `app-1` on 3000 and `app-2` on 9000 is fine. Every container runs the same image and gets an `INSTANCE` variable of its own. Start, stop, restart and remove act on the whole set.

A port that is already taken on the server is caught as you type it. The panel checks the chosen server for another of your services on that port and for anything else already listening on it, marks the row, and says which one holds it — for example "*Port 3000 is already used on web-1 by shop-api*". The same check runs again when you press **Build & deploy** and when you change ports on an existing service, so the deploy never starts on a port it cannot have. The same port typed on two rows is refused too.

The same list is on each card afterwards, under **Containers**: add one, remove one or move one to another port and press **Apply**. The containers are recreated from the image already on the server, so this takes seconds — nothing is cloned, nothing is rebuilt and no code changes. A service that was stopped stays stopped, with the new shape waiting for whenever it is started.

Then it builds and runs it

The deployment happens in one pass over SSH: clone the branch into `/opt/auto-deploy/apps/<name>`, write a Dockerfile (unless the repository has one you chose to use), `docker build` once, then replace the containers. **The previous release keeps running until the new image has built**, so a broken commit leaves the site up.

With **Use the Dockerfile in the repository** ticked, your Dockerfile decides how the app is installed, built and started, so the install and build commands are greyed out and no build or start command is asked for — an Angular or React project with its own Dockerfile deploys without one. Set **Port inside the container** to the port your Dockerfile's app listens on.

The form also carries the port inside the container, volumes, environment variables, the Docker network to join and the restart policy.

Volumes

Every deploy replaces the container, and **anything written inside a container goes with it** — uploads, a SQLite file, generated thumbnails. A named volume is where those survive, so the form has a **Volumes** list: a row per volume, each with its name and where it is mounted inside the container.

The default is filled in for you. A server-side project arrives with `<name>-data` mounted at `/app/data` already on the list — the image the panel writes works in `/app`, so that is where an app's own data belongs. Remove the row if the service keeps nothing, or press **Use the default** to put it back. A static site gets none, because it is built into the image and served by nginx with nothing to keep.

Nothing is created up front: Docker makes a named volume the first time it is mounted and finds the same one on every deploy after that, which is exactly what makes the data outlive the container. The mount path is checked — absolute, no traversal, and never over `/`, `/etc`, `/usr` or the container's other system directories, which would break the image. Up to six per service.

When a service runs **more than one container**, the note under the list says so: all of them mount the same volume, which is what you want for uploads and very much not what you want for a SQLite file.

Removing a service asks about its volumes separately and **keeps them unless you say otherwise** — they are the one part of a service that cannot be rebuilt from the repository. Kept volumes are found again if you deploy the same service back onto that server.

Environment variables

The environment box takes `KEY=value`, one per line — or **Import a .env file**, which reads a file from your machine and drops it in **exactly as it is**: comments and blank lines are kept in the box and skipped when it is parsed, `export KEY=value` is understood, and a quoted value keeps its spaces. A key that appears twice keeps the last one, as a shell would. Nothing is uploaded anywhere; the file is read in the browser. `PORT` and `INSTANCE` are set for every container by the panel and are refused here, so they cannot fight with the port mapping.

The variables are there during the build too, not only while the app runs. A Next.js, Vite or React build often reads them (`NEXT_PUBLIC_API_URL`, `VITE_API_URL`, a database URL used while pages are pre-rendered), and a build that cannot see them fails. The build step gets them for that one command only: they are never written into the image, so pulling or inspecting the image does not reveal them. Values a framework deliberately bakes into the browser bundle (`NEXT_PUBLIC_*`, `VITE_*`) are of course in the built files, exactly as on any other host. A repository with its own Dockerfile can opt in by starting its build line with `RUN --mount=type=secret,id=appenv` and loading `/run/secrets/appenv` (`set -a; . /run/secrets/appenv; set +a; npm run build`).

A domain for the app

Tick **Add a domain** on the form (or press **+ Add domain** on the app later) and choose how it is published:

Option	Needs	What the panel does
DNS — nginx + free SSL certificate	a connected Cloudflare account; ports 80 and 443 open	creates the DNS record, an nginx site and a certificate
Zero Trust — Cloudflare Tunnel	a connected Cloudflare account	adds the hostname to a tunnel; no open ports
Direct — nginx (your own DNS)	nothing on Cloudflare; you point the domain yourself	writes an nginx site in front of the app and, if **SSL required** is ticked, gets a free certificate

Direct is for domains that are not on Cloudflare — managed at GoDaddy, Namecheap, Hostinger or anywhere else. Type the full hostname (`shop.example.com`) and, at your DNS provider, add an **A record** for it

pointing at the server's public IP address. With **SSL required** ticked, the panel waits up to two minutes for that record to be visible, then gets a Let's Encrypt certificate and redirects HTTP to HTTPS; if the record is not there yet the domain shows an error that says so, and **Retry** picks it up once it is. Without SSL, the app answers on plain `http://` right away. Ports 80 (and 443 for SSL) must be open on the server.

The same thing can be done from the server side: on a server's **Nginx** tab, **+ Add domain → A custom app on AJ Pilot** — see Domains below.

When a build fails, the message on the card quotes the error lines from the build itself (a type error, a missing module, a missing variable) rather than Docker's closing summary. The full build log is under **Deploy log** on the card.

Afterwards, **Environment** on a card opens the variables as they are — a row each, with the value shown so it can be corrected rather than retyped. Add a row, remove a row, or import another `.env` file to merge it in (a key in the file replaces the one on screen). **Restart the containers now** is ticked by default: Docker fixes a container's environment when it is created, so the containers are recreated from the image already on the server for the change to take. Untick it and the values are saved and picked up at the next deploy. Only a role with the **edit** permission may open this — it is the one place the panel shows a stored secret in the clear; everyone else sees the key names on the card and nothing more.

Docker Hub is optional

Tick **Push the image to Docker Hub** and pick a saved account: the image is tagged `user/app:tag`, pushed, and then run. Leave it off — the default — and the image is built and run **only on that server**; nothing leaves the machine.

Image names are lowercase-only, so the account's username is lowercased into the image name — a Docker Hub username with capitals used to fail at the build with nothing but `invalid reference format` to go on. The sign-in is checked separately from the push, so a rejected token and a refused push say so in different words, and the token needs **Read & Write**: a read-only one can pull but never push. With no Docker Hub account saved the tick box says so and offers to add one rather than sitting there refusing to be ticked.

Deploying does not hold you up

A build takes minutes, so the panel does not make you watch a spinner in a dialog. Pressing **Build & deploy** hands the job to the server, the popup closes at once — the request comes back in well under a second — and the app appears in the list as a card marked **in progress**, with a moving bar and a counter of how long it has been going. The card refreshes itself every few seconds and settles on **running** or **error** when the server is done; the poll stops as soon as nothing is building, and when you leave the page.

Because the build runs on the server rather than inside the request, closing the tab does not cancel it. Asking for a second deploy of the same app while one is in flight is refused rather than queued, and a deployment left hanging by a panel restart is released after 40 minutes.

Each card is badged with its **project type** next to the status — ▲ Next.js, Angular, React · Vite, NestJS, Node.js service, Static site — and names the repository, the branch and the **server it runs on** in its heading, with that server's name and host again under **Running on**. Underneath it says how it runs: `served by nginx` or `Node 22`, how many containers, and whether the image went to Docker Hub. Every container is listed with the port it answers on.

Afterwards each app has **Redeploy** (rebuilds the current branch), **Logs** (the container's output, plus the build log when a deployment failed), **Environment**, **Containers**, start/stop/restart, and **Remove**, which deletes the containers and the clone.

The same cards on the server page

A server's **Apps** tab lists everything deployed onto that machine, using those same cards — the tab count is how many there are. Every button works there: **Redeploy**, **Logs**, start/stop/restart and **Remove**, with the in-progress card refreshing itself exactly as it does on the Apps page. **+ Custom service** on the tab opens the deploy wizard with that server already picked, and when the build starts you stay on the server page with its Apps tab in front of you rather than being taken to the Apps list.

The clone URL carries the git token, so it is passed to the server base64-encoded, masked out of git's own output, and scrubbed from every log line before it is stored or shown.

Nginx and SSL

Every server has an **Nginx** tab. It reads the machine's own nginx — the panel keeps no copy of anything, and a domain is a file in `sites-available` (or `conf.d` on a server whose `nginx.conf` does not include `sites-enabled`). The file on the server is the truth, and the tab reads it back every time.

If nginx is not installed, the tab says so and offers to install it: `nginx` from the distribution's packages, enabled and started, and — only if ufw is actually active — `Nginx Full` allowed through it.

If it is, the tab opens with a status header that says in one line whether nginx is healthy — running or stopped, the version, the ports it listens on, whether the configuration test passes and whether certificates renew themselves — with **+ Add domain**, **Test config** and **Reload** beside it. Below it, four counters: domains, how many are secured with SSL, how many **need attention** (no SSL, a certificate expiring within 14 days, or disabled) and certificates. When `nginx -t` does not pass, that comes first in red with the output, because nothing else on the tab can be trusted until it does; a reload is refused if the configuration does not pass, so a broken edit cannot take the site down.

Domains

Every server block on the machine is shown as a card — the ones this panel wrote and the ones that were already there. Each card has a 🔒 (secure) or ⚠ (not secure) marker, the domain as a link that opens it, its other names, **where it points** — named by app when it forwards to one of your AJ Pilot apps on this server, otherwise the address or the folder — **how many days its certificate has left**, whether it is enabled, and the file it lives in. A domain without SSL says so on its card with an **Add SSL** button. A search box and **All / Secure / Not secure / Disabled** filters narrow a long list. With no domains yet, the tab explains the three ways to add one.

+ Add domain opens a dialog that starts with three choices: **AJ Pilot app** (an app deployed by the panel), **Port on this server** (anything listening on a port — WebSockets passed through, the usual `X-Forwarded-*` headers set) or **Static folder** (files on disk, with a single-page fallback if it needs one). Then the domain names, then **SSL required**. The dialog shows the server's address, which the domain's A record must point at. Upload size, WebSockets and enabling the site are under **Advanced**. **Edit** reopens it — as the form for a site the panel wrote, or as the file itself, which is what you get for a site nginx already had or one certbot has rewritten, because regenerating those would throw away their HTTPS block. **Enable** / **Disable** switches the

`sites-enabled` link without touching the file, and **Remove** deletes it. The certificate is kept when a domain is removed, so adding it back does not mean asking Let's Encrypt for a new one.

SSL required on the add form gets a certificate as soon as the site is written, the same as pressing **Add SSL** afterwards — the domain must already point at this server. Leave it unticked for a site that stays on plain HTTP, or one whose DNS is not ready yet.

A custom app on AJ Pilot is the third choice under *What it serves*. Pick the **server** and then one of the **apps** deployed on it (and which of its containers, if it runs several); the domain is attached to that app, so it appears on the app's page with its status and setup log, can be retried there, and is removed with the app. nginx and, with **SSL required**, certbot are installed if missing. No Cloudflare account is needed — point the domain's A record at the server yourself.

No write is ever trusted. Each one keeps a backup, writes the file, runs `nginx -t`, and — if the test fails — puts back exactly what was there before reporting the `[emerg]` line. nginx is only reloaded once the test has passed.

Load balancing (upstreams)

An **upstream** is a group of servers that share the traffic of a domain — two copies of an app on ports 3000 and 3001, say, so one can restart while the other answers. The **Load balancing** section near the bottom of the tab shows each one as a card:

- a health pill — how many of its servers answer right now — and how traffic is shared, in plain words;
- every server with a green or red dot, the app it belongs to when it is one of your AJ Pilot apps on this server, and a bar showing **what share of the traffic it gets**; backups and paused servers are marked;
- the domains that use it — or, when none does yet, the line to put in a domain to use it.

+ Add upstream walks through three steps: a **name**; the **servers**, where one click adds any of your apps or any port already listening on this server (or type `host:port`); and **how traffic is shared**:

Choice	What it does
Take turns	each server in order (nginx's round robin) — the default
Least busy	the server with the fewest open connections
Same visitor, same server	by visitor IP, so a visitor keeps landing on one server
By a key	by a value you choose, such as a cookie
Random	any server

Each server has a **weight** (its share of traffic), **Backup** (only used when the others fail) and **Paused** (taken out without deleting it). Failure limits, timeouts, connection caps and keepalive are under **More**. As before, the file is tested with `nginx -t` and put back if the test fails.

Ports & proxies, folded below it, lists every place nginx forwards to and every port bound on the server.

Certificates

Load balancing, ports and certificates are folded into their own sections at the bottom of the tab, each showing its count; open one to see it in full. The **SSL certificates** section's header turns yellow when a certificate expires within 14 days and red when one has expired.

Inside, a status strip shows certbot's version (and whether its nginx plugin is there), whether **automatic renewal** is on — with a **Turn on** button when `certbot.timer` is not active — and how many certificates are healthy, expiring soon or expired. When any need attention, a banner says so with **Renew everything due**.

Each certificate is a card, soonest expiry first: a large **days left** with a bar showing how much of its 90-day life remains (green, yellow under 14 days, red once expired), the expiry date, its domains as links, **which nginx domains use it**, where its files are, and a **Renew** button. **+ Get a certificate** is at the top. **If certbot is not installed**, the section explains what it is for and one button installs it with its nginx plugin and enables the renewal timer.

Add SSL on a domain (or **+ Get a certificate**) opens a dialog with a short **before you start** checklist — the domain's A record must point at this server's address, which it shows, and port 80 must be open — then the domain names (with one-click chips for every domain on this server that is not secured yet), an email for expiry notices and a **Redirect HTTP to HTTPS** switch. It runs `certbot --nginx`, which proves you control the domain over port 80 and writes the certificate into that vhost. Under **Advanced**, the **staging** option asks Let's Encrypt's test service instead, which is the way to try a setup without spending one of the real rate-limited issuances.

Renew runs `certbot renew` for one certificate, and **Renew everything due** for all of them; either way nginx is reloaded afterwards and the panel says plainly when nothing was due yet. The failures that actually happen are translated: DNS that does not point here, port 80 not reachable from the internet, the rate limit hit, or no matching server block for that domain.

Everything on this tab runs as root over the existing SSH connection. Domain names, site names and certificate names are validated against strict patterns before they are ever put in a command.

Scheduled jobs

A server's **Cron** tab is everything that machine runs on a schedule. **If cron is not installed** it says so — "*Nothing here runs on a schedule until it is — no backups, no cleanups, no certificate renewals*" — with a button to install it and start its service. If the service is installed but **stopped**, that comes first in red, because jobs sitting in a table nobody reads are worse than no jobs at all.

Cron keeps its work in four places, and the tab reads all of them:

Where	What it is	Editable here
a user's crontab	what `crontab -e` edits, one table per user	**yes**
`/etc/crontab`	the system table, with a user column	no
`/etc/cron.d/*`	files dropped in by packages	no
`/etc/cron.{hourly,daily,weekly,monthly}`	scripts run by `run-parts`	no

Add, **Edit** and **Delete** work on user crontabs. The rest is shown in full but left alone: those files belong to the distribution and its packages, and quietly rewriting them from a web panel is how a server stops doing what its owner expects.

Every job is read back in plain English next to its schedule — `*/15 * * * *` becomes "*every 15 minutes*", `0 4 * * 0` becomes "*every Sunday at 04:00*" — and the add form does the same as you type, so a wrong schedule is obvious there rather than at three in the morning. There are presets for the usual cadences.

Nothing is written carelessly. An edit is applied by reading the whole table, changing the one line, and handing it back to `crontab`, which parses it and **refuses the lot if any line is wrong** — so a rejected edit leaves the server exactly as it was, and the refusal is what you are shown. The job being changed is matched on its exact line, so an edit made from a stale page fails loudly instead of overwriting whatever is there now. The schedule and command are checked before any of that: five fields or one of cron's own names, one line only, and an unescaped `%` is refused with the reason, because cron reads it as "the rest is stdin" and it is the classic way a job silently does nothing.

The tab also shows what cron most recently logged, which is usually the fastest answer to "did it run?".

CI runners

A self-hosted runner ties together three things the panel already knows: a **git account** (who it registers with), a **repository or organisation** (what it builds) and one of your **servers** (where it runs).

Open a git account and use **+ Add runner**:

1. Pick the account, then a repository or a whole organisation/group.
2. Pick the server — the list is the servers you have already added.
3. Name it, give it labels, and choose which user the service runs as.

The panel then mints a registration token from GitHub or GitLab, installs the runner over that server's existing SSH connection, and keeps it running under systemd. A first install downloads the runner on the server, so it takes a minute or two; the install log comes back either way.

Provider	What is installed
GitHub	`actions/runner` (latest release) into `/opt/auto-deploy/runners/<name>`, registered with a repository or organisation token, then `svc.sh install` writes its own systemd unit
GitLab	the `gitlab-runner` package (once per server), registered against the project or group with the `shell` or `docker` executor
Bitbucket	not supported — Bitbucket Pipelines runners work differently, so Bitbucket accounts are left out of the runner form. They can still deploy apps.

Once installed, a runner can be checked (live status from the provider), restarted, stopped and removed. Removing it unregisters it at the provider, deletes it from the server and forgets it here.

The git account page also shows runners that were **not** installed from this panel — it checks the account's most recently updated repositories — and each repository page lists that repository's own runners.

Is a runner actually running?

A server's **Runners** tab asks *that machine*, not the provider, and the two answer different questions. A runner whose service has died still sits in GitHub's list; a runner somebody installed by hand is not in the panel's list at all. So the tab opens with the sentence you want — "*This runner is running on this server*", or "*2 of 3 runners are running*", or "*No runner is running on this server*" in red — and says whether one is **working on a job right now**, which it knows because GitHub's runner forks a `Runner.Worker` process for each job it picks up.

Each row then shows **running here** as the machine reports it: active or not, since when, whether it starts at boot, and its pid — beside what the provider last thought, kept in a separate column so the two are never confused. A runner whose service is not on the machine at all says so rather than showing a stale status, and the stored status is brought back in line with what the server just said. Stopped runners get a **Start** button; running ones get **Restart** and **Stop**. Runner services found on the machine that no runner of yours accounts for are listed underneath, the same way the Docker tab lists containers it did not install.

The tab's counter is how many are running, and turns red when any of them is stopped or missing.

Databases (MySQL, PostgreSQL, MongoDB, Redis, Elasticsearch)

The **Databases** tab connects to the MySQL, PostgreSQL, MongoDB, Redis and Elasticsearch servers running on your machines. Click **+ Add database connection** and pick the engine; the port, the auth-database field (MongoDB) and the TLS box (PostgreSQL, MongoDB, Redis, Elasticsearch — for Elasticsearch it means HTTPS) follow the choice. MongoDB, Redis and Elasticsearch may have no password. A MongoDB `mongodb+srv://…` string also works as the host, for Atlas. The chips above the list filter it by engine.

A database installed from **Installations** (MySQL, PostgreSQL, MongoDB, Redis, Elasticsearch) is added here by itself, tunnelled through its server with the superuser it was created with.

PostgreSQL, MongoDB and Redis get the same four tabs as MySQL (Elasticsearch is described after the table):

	PostgreSQL	**MongoDB**	**Redis**
Overview	version, uptime, databases, users, connections, cache hit rate, transactions, rows written, deadlocks, sessions	version, uptime, databases, users, connections, opcounters, memory, WiredTiger cache, traffic, replica role, current operations	version, uptime, keys, ACL users, clients, memory vs `maxmemory`, ops/sec, hit rate, evictions, persistence, role, client list
Databases	create (owner, encoding), drop (optionally forcing sessions off); per database: size, tables, live/dead rows, indexes, schemas, extensions, who can connect; per table: columns, indexes, first rows	create (with a first collection), drop; per database: collections, documents, sizes, users with roles on it; per collection: indexes, first documents	db0–db15 with key counts, **Flush**; per database: a SCAN sample of keys with type, TTL and memory; per key: its value
Users	roles with login, superuser, connection limit, expiry, sessions and grants. Create, grant read-only / read-write / full / owner on a database, revoke, password, lock (NOLOGIN), drop. Dropping hands anything the role owned to the panel's role first, so no table is deleted	users with auth database and roles. Create, grant a role (per database or `…AnyDatabase` / `root`), revoke every role on a database, password, drop. MongoDB has no account lock	ACL users. Create with full / read-write / read-only access on a key pattern, change access, revoke all commands, password, enable / disable, delete. Saved with `ACL SAVE` or `CONFIG REWRITE` when the server has a file for it
Configuration	20 settings via `ALTER SYSTEM` + reload (persistent; "restart" marks the ones that need one), plus all of `pg_settings`	runtime `setParameter` values and the profiler's `slowms` (lost on restart), plus every parameter	13 settings via `CONFIG SET`, optionally saved with `CONFIG REWRITE`, plus every setting

	PostgreSQL	MongoDB	Redis
Run query	runs inside a `READ ONLY` transaction that is rolled back, as a single prepared statement	JSON find or aggregate; `$out` / `$merge` refused	one command from a read-only allow-list

Elasticsearch

An Elasticsearch connection uses the HTTP API (port 9200). Sign in as `elastic` or another user with the `superuser` role, or leave the username and password empty if security is turned off. Tick **TLS** when the cluster answers on `https://`; self-signed certificates are accepted. Its page has the same tabs, with **Indices** in place of Databases:

- **Overview** — version, cluster health (green / yellow / red), nodes, indices, documents, size on disk, active and unassigned shards, JVM heap, and a table of nodes with their roles, heap, RAM, CPU, load and disk
- **Indices** — every index with its health, status, documents, size and shard counts. **+ Create index** asks for the name and the number of shards and replicas (keep replicas at 0 on a single node, or the cluster turns yellow). **Delete** asks you to type the name back. Indices whose names start with `.` belong to Elasticsearch and cannot be deleted here. Click an index for its documents, deleted documents, size, shards, aliases, settings, its fields with their types, and its first 20 documents. Click a field to see how many documents have it and, for keyword, number, date and boolean fields, its 20 most common values
- **Users** — security users with their roles and whether they are enabled. **+ Create user**, change **Password**, **Disable** / **Enable**, **Delete**, **Grant** a built-in or custom role or read / write / full access to an index pattern such as `logs-*`, and **Revoke** a role. Built-in users such as `elastic` cannot be deleted, and the user the panel signs in as cannot disable itself. On a cluster with security off this tab says so
- **Configuration** — persistent cluster settings (shard allocation, disk watermarks, recovery speed, the default search timeout, shards per node, automatic index creation, `action.destructive_requires_name`). They survive restarts. Every other setting is listed below them
- **Run query** — the same syntax as Kibana Dev Tools: a first line such as `GET /_cat/indices?v` or `POST /orders/_search`, then an optional JSON body. A JSON body on its own searches the index in the Database box. Search hits show as rows with their fields as columns. Only reads are allowed: `GET`, and `POST` to `_search`, `_count`, `_msearch`, `_field_caps`, `_validate/query` and `_sql`

The rest of this section describes the MySQL page.

Most MySQL installs bind to `127.0.0.1`, so nothing outside the box can reach them. Rather than asking you to open port 3306 to the world, a MySQL connection can be **tied to a server** — the panel then opens the database connection *through that server's existing SSH connection*. No firewall change, no extra exposure. Pick the server in the "Connect through server" dropdown; leave it on *Direct connection* for a database that is already reachable.

A connection can be changed later with **Edit** (on its card, or **Edit connection** on its page); leave the password empty to keep the saved one. Once connected, the page has four tabs:

- **Overview** — statistics: version, uptime, databases, rows, total users (active / locked), connections vs. `max_connections`, queries per second, slow queries, buffer pool, traffic, binlog and read-only state; the connection details, the largest tables and the current process list

- **Databases** — every schema with its tables, approximate rows, data and index size and charset. **+ Create database** (optionally with a user that owns it), **Charset** to change the default character set / collation, and **Drop**, which asks you to type the name back. Click a database for its statistics — tables, columns, rows, size, engines, routines, triggers, events, open sessions — and the accounts that can reach it (granted on that database, or globally); click a table for its columns, indexes and foreign keys
- **Users** — user management: totals (all, active, locked, password expired), each account with its auth plugin, state, open sessions and full `SHOW GRANTS`. **+ Create user**, **Grant** privileges on one database or all of them, **Revoke**, change **Password** and connection limit, **Lock** / **Unlock**, and **Drop**. The account the panel itself signs in as cannot be locked or dropped from here
- **Configuration** — the settings that matter (connections, timeouts, memory, logging, SQL mode, time zone, character set) with what each does, editable in place with `SET GLOBAL` — or `SET PERSIST` on MySQL 8 so it survives a restart — plus every server variable, searchable. Only that curated list can be changed
- **Run query** — a read-only console

Viewers can see all of it. Creating databases, users and grants needs the *create* permission, changing a charset, password, lock or setting needs *edit*, and dropping or revoking needs *delete*. Every change goes to the activity log. The MySQL account in the connection needs the matching MySQL privileges too (`CREATE`, `DROP`, `CREATE USER`, `GRANT OPTION`, `SYSTEM_VARIABLES_ADMIN`); when it lacks one, MySQL's own error is shown.

The server detail page also shows MySQL found on the host itself (version, service state, bind address, data directory size), so you can see a database exists before you connect to it.

The query console is read-only

Only `SELECT`, `SHOW`, `DESCRIBE` and `EXPLAIN` run. Anything else — `INSERT`, `UPDATE`, `DELETE`, `DROP`, stacked statements after a `;`, `INTO OUTFILE`, `LOAD_FILE`, `SLEEP`, `BENCHMARK` — is refused before it reaches MySQL. Results are capped at 200 rows and queries time out after 20 seconds.

This is a guard against accidents, not a security boundary: it constrains what this panel will send, and the MySQL account's own grants remain what actually protect the data. Give the panel a read-only MySQL user if you want that enforced by the database itself.

Git accounts

Three git hosts are supported: **GitHub** (github.com or Enterprise), **GitLab** (gitlab.com or self-hosted) and **Bitbucket** (bitbucket.org). All three can be connected by signing in through the browser or by pasting a token, and a connected account of any of them can deploy apps. Runners are GitHub and GitLab only — see CI runners.

	GitHub	GitLab	Bitbucket
Browser sign-in	OAuth app	OAuth application (with PKCE)	OAuth consumer
Paste a token	✓	✓	✓
Repositories, branches, commits	✓	✓	✓
Deploy apps (clone with the token)	✓	✓	✓
Organisations / groups / workspaces	organisations	groups	workspaces

	GitHub	GitLab	Bitbucket
CI runners	GitHub Actions runner	`gitlab-runner`	—
Self-hosted server	GitHub Enterprise	self-hosted GitLab	— (Bitbucket Cloud only)

Connecting through the browser

Click **+ Add git account**, pick GitHub, GitLab or Bitbucket, and you are sent to their sign-in page. You choose the account there and approve access; the panel receives a short-lived code, swaps it for an access token server-side, fetches the account behind it, and stores it. Nothing is typed in by hand, and the token never appears in a URL or in the browser.

First time only: registering the app

No provider lets anyone sign in until an OAuth app exists for the panel, so the very first time you pick a provider the panel walks you through it in two steps:

1. **Open the provider** and create the app, pasting the **callback URL** the wizard shows (it has a copy button). It is always `<OAUTH_CALLBACK_BASE>/api/git/oauth/callback`, e.g. `http://localhost:4000/api/git/oauth/callback`.
2. **Paste the client ID and secret back** — the panel saves them to `.env` and starts using them immediately. No restart, and you never edit a file by hand.

What to do on each provider:

	GitHub	GitLab	Bitbucket
Where	Settings → Developer settings → OAuth Apps → *New OAuth App*	Preferences → Applications → *Add new application*	your workspace → Settings → OAuth consumers → *Add consumer*
What to do	the form opens with the name and callback already filled in — press *Register application*, then *Generate a new client secret*	paste the callback URL and tick the scopes below	paste the callback URL, tick *This is a private consumer*, and tick the permissions below
Access it gets	scopes `repo`, `read:org`, `read:user` (requested at sign-in)	scopes `read_api`, `read_repository`, `read_user`	permissions **Account: Read**, **Workspace membership: Read**, **Repositories: Read** (set on the consumer)
Client ID is called	Client ID (`Ov23li…` or `Iv1.…`)	Application ID (long hex string)	Key
Client secret is called	Client secret	Secret	Secret
Access token lifetime	does not expire	2 hours, renewed automatically	2 hours, renewed automatically

Before saving, the panel checks the pair against the provider's token endpoint. A Client ID the provider has never heard of, or a secret that does not match it, is rejected there and then with the reason — rather than

sending you to a sign-in page that answers **404**, which is what GitHub does for an unknown app. Your username, email or password are *not* the Client ID; it comes from the app you register.

After that, adding an account is just: pick provider → sign in → done.

The flow is protected with a single-use `state` value (rejected on replay or after 10 minutes), and GitLab additionally uses PKCE. GitLab and Bitbucket access tokens expire after two hours; the refresh token is stored encrypted and the panel renews the access token automatically before it lapses, so a connection keeps working.

Git settings in `.env`

None of these are needed to run the panel — they are left out of `.env.example` on purpose. Without them, browser sign-in is simply not set up yet and the wizard offers to set it up; pasting a token always works. The wizard writes the `*_CLIENT_ID` / `*_CLIENT_SECRET` lines (and `OAUTH_CALLBACK_BASE`) to `.env` for you when you register an app. To set them by hand instead, add any of these and restart:

```
# Where the provider sends the browser back after sign-in. Set it when the panel
# is reached on another host or port, and register that host's callback instead.
OAUTH_CALLBACK_BASE=http://localhost:4000

# GitHub OAuth app — Settings -> Developer settings -> OAuth Apps -> New OAuth App
GITHUB_CLIENT_ID=
GITHUB_CLIENT_SECRET=

# GitLab application — Preferences -> Applications (read_api, read_repository, read_user)
GITLAB_CLIENT_ID=
GITLAB_CLIENT_SECRET=

# Bitbucket OAuth consumer — workspace -> Settings -> OAuth consumers -> Add consumer
# (Account: Read, Workspace membership: Read, Repositories: Read). The consumer's Key is the client
ID.
BITBUCKET_CLIENT_ID=
BITBUCKET_CLIENT_SECRET=

# Self-hosted GitHub Enterprise / GitLab only
# GITHUB_WEB_URL=https://github.example.com
# GITHUB_API_URL=https://github.example.com/api/v3
# GITLAB_WEB_URL=https://gitlab.example.com
# GITLAB_API_URL=https://gitlab.example.com/api/v4
```

Variable	Default	Purpose
`OAUTH_CALLBACK_BASE`	`http://localhost:<PORT>`	base URL the provider redirects back to
`GITHUB_CLIENT_ID` / `GITHUB_CLIENT_SECRET`	—	OAuth app for GitHub browser sign-in
`GITLAB_CLIENT_ID` / `GITLAB_CLIENT_SECRET`	—	OAuth application for GitLab browser sign-in
`BITBUCKET_CLIENT_ID` / `BITBUCKET_CLIENT_SECRET`	—	OAuth consumer (Key / Secret) for Bitbucket browser sign-in

Variable	Default	Purpose
`GITHUB_WEB_URL` / `GITHUB_API_URL`	`https://github.com` / `https://api.github.com`	self-hosted GitHub Enterprise
`GITLAB_WEB_URL` / `GITLAB_API_URL`	`https://gitlab.com` / `https://gitlab.com/api/v4`	self-hosted GitLab

If you reach the panel on a different port or host, set `OAUTH_CALLBACK_BASE` to match before registering — the callback URL the wizard shows is the one that must be registered with the provider.

Pasting a token instead

There is a small *Paste an access token instead* link on the picker, for a CI token or a machine account where browser sign-in makes no sense. Pick the **Hosting** (GitHub, GitLab or Bitbucket), paste the token and press **Authenticate** to check it before saving.

	GitHub	GitLab	Bitbucket
Token	personal access token, classic or fine-grained	personal access token	Atlassian API token, or a workspace / repository access token
Scope needed	`repo`	`read_api` (plus `read_repository`)	read access to account, workspaces and repositories
Create it at	Settings → Developer settings → Personal access tokens	Preferences → Access tokens	Atlassian account → Security → API tokens; or workspace / repository settings → Access tokens
Extra field	API URL (Enterprise only)	API URL (self-hosted only)	**Atlassian email** — fill it in for an API token, leave it empty for an access token

How the token is used for Bitbucket: with an email it is sent as `email:token` (basic auth) and git clones as `x-bitbucket-api-token-auth`; without one it is a bearer token and git clones as `x-token-auth`. An older Bitbucket *app password* also works: put your Bitbucket username in the email field.

(Those tables are for **Paste a token**; browser sign-in gets its access from the app you registered.)

What you see for each account

- **Identity** — login, display name, avatar, account type, company, and what kind of token it is (a classic GitHub token with its scopes listed, fine-grained, OAuth, or a Bitbucket API / access token)
- **API budget** — requests left in the current rate-limit window (GitHub)
- **Repositories** — everything the token can reach, with visibility, default branch, language and last push
- **Drill-down** — click a repository for its branches (protected ones flagged on GitHub and GitLab), click a branch for its recent commits
- **Organisations** — GitHub organisations, GitLab groups or Bitbucket workspaces the account belongs to

`Re-authenticate` re-checks a token and refreshes the stored account — useful after rotating a token or changing its scopes.

Credentials

The **Credentials** tab stores access for the deployment phase. Each secret is encrypted before it is written to disk and never leaves the API — the UI only ever sees a masked hint.

Provider	What to supply	Verified against
Git	GitHub / GitLab / Bitbucket access token (see **Git accounts** above)	`GET /user`
Docker Hub	username + access token	`POST /v2/users/login`
Cloudflare	browser sign-in (**Account management** → **Cloudflare**) or an API token with `Zone:Read` + `DNS:Edit`	`GET /user/tokens/verify`, then `/user`, `/accounts`, `/zones`
MySQL	user + password, host/port, optional server to tunnel through	`SELECT VERSION()` over the real connection

Verify calls the provider's own API, so nothing is assumed valid until it is checked.

Git, Cloudflare and Docker Hub accounts all live under **Account management** in the sidebar, one tab each. MySQL connections stay under **Databases**.

Cloudflare browser sign-in. Cloudflare does not offer OAuth to self-hosted panels, so **+ Connect Cloudflare** opens your Cloudflare dashboard with a *Create API token* form already filled in (name, Zone:Edit, Zone Settings:Read, DNS:Edit, Account Settings:Read, User Details:Read, all zones). Sign in there, press *Continue to summary* → *Create Token* → *Copy*, and come back: the panel reads the token from the clipboard (or from the paste box), verifies it, then fetches and stores your email, accounts and zones. Open the account to browse zones and each zone's DNS records.

Security notes

- Every secret — SSH passwords, private keys, passphrases, provider tokens — is encrypted with **AES-256-GCM** before it is written to MySQL. No secret is ever returned by the API, and anyone reading the database directly sees only ciphertext.
- The encryption key comes from `APP_ENCRYPTION_KEY`, or from `data/master.key` if that variable is unset. **Back this key up.** Losing it makes every stored credential unreadable.
- `data/` and `.env` are gitignored. Do not commit them.
- The panel signs people in and keeps organisations apart at the query level: every read and write is filtered by the organisation of the session making it, so one organisation cannot see or touch another's servers even by guessing ids. Still put it behind HTTPS before exposing it, and set `AUTH_SECURE_COOKIE=true` when you do.
- Passwords are hashed with **scrypt**; session cookies hold a random token whose only stored form is a SHA-256 hash. Roles are checked by the API on every mutating request, not by the buttons.
- Creating services and installing runners runs commands as **root** on the target server, via `sudo` when the SSH user is not root. The sudo password, if one is needed, is encrypted like every other secret and is only ever sent on the command's stdin. Unit names and service names are validated against a strict pattern rather than escaped, and every generated script is passed base64-encoded so nothing typed into a form can break out of it.
- Only units in `/etc/systemd/system` that this panel created can be deleted through it.

- Container names, network names, image versions and ports are all validated against a strict pattern before they reach a command line, and passwords for installed services are encrypted at rest like every other secret. Removing an installation keeps its data volume unless you explicitly say to delete it.

Configuration

Copy `.env.example` to `.env` to override any of:

Variable	Default	Purpose
`PORT`	`4000`	HTTP port
`HOST`	`0.0.0.0`	bind address
`DB_HOST`	`127.0.0.1`	MySQL host for the panel's own data
`DB_PORT`	`3306`	MySQL port
`DB_USER`	`root`	MySQL user
`DB_PASSWORD`	*(empty)*	MySQL password
`DB_NAME`	`auto_deploy`	database name, created if missing
`DB_POOL_SIZE`	`10`	connection pool size
`APP_ENCRYPTION_KEY`	generated	master key for secret encryption
`SSH_CONNECT_TIMEOUT`	`15000`	SSH handshake timeout (ms)
`SSH_EXEC_TIMEOUT`	`45000`	remote command timeout (ms)
`HEALTH_CHECK_ENABLED`	`true`	background online/offline sweep
`HEALTH_CHECK_INTERVAL_SECONDS`	`5`	how often every server's SSH port is probed
`HEALTH_CHECK_TIMEOUT`	`4000`	a probe slower than this counts as offline (ms)
`HEALTH_CHECK_CONCURRENCY`	`10`	servers probed at the same time
`HEALTH_CHECK_SIGNIN_MINUTES`	`10`	minutes between actual sign-in checks
`HEALTH_CHECK_SIGNIN_TIMEOUT`	`12000`	how long a sign-in check may take (ms)
`STATS_MAX_STREAMS`	`8`	live views that may be open at once
`STATS_STREAM_MINUTES`	`15`	how long one live view runs before the browser reconnects

Connection monitor

The panel watches every server in two tiers, because *reachable* and *usable* are different questions that cost very different amounts to answer.

Every 5 seconds it opens a TCP connection to each server's SSH port and closes it again — no key exchange, no login, a few milliseconds per server — and that is what the list's online/offline state means. The answers are kept in memory and written to the row only when a server actually **changes** state, so watching a fleet every five seconds does not mean a database write per server per five seconds. A change is also the only thing that reaches the activity log, as `went_online` or `went_offline`, so a host that stays up does not fill the feed.

Every 10 minutes it actually signs in to each reachable server and runs one command. That is the only thing that proves the stored credentials still work, and it is far too expensive to run on the fast cadence. A server whose port answers but whose sign-in fails is shown as **online** with a **cannot sign in** badge and the reason — which is the truth, and a different problem from being down.

The servers list polls `/api/servers/status` on the same 5-second cadence while it is open. That endpoint is deliberately tiny — statuses, latency and when each was checked, with no system profiles and no joins — and the cards are patched in place, so a card you are using is never redrawn under your cursor.

When it last updated is said in three places. Above the list, "*Last updated 14:22:07 · checking every 5s*" — an absolute time, so a page that has quietly stopped refreshing is obvious rather than frozen at a plausible-looking "3s ago". On each card, how long the port took to answer and how long ago that was, with the exact time on hover, plus when the panel last signed in. And on a server's own page, the same line under its name, kept up to date while you are on it.

API

Method	Path	Purpose
`GET`	`/api/health`	counts for your organisation, database in use, uptime
`GET`	`/api/auth/state`	needs setup, who is signed in, the role table
`POST`	`/api/auth/setup`	first run only: create the super admin and the first organisation
`POST`	`/api/auth/login`	sign in
`POST`	`/api/auth/logout`	sign out
`PUT`	`/api/auth/profile`	change your own name or sign-in email
`POST`	`/api/auth/password`	change your own password (ends your other sessions)
`POST`	`/api/auth/organisation`	super admin: work in another organisation
`GET`	`/api/team/members`	the people in your organisation, and the roles you may hand out
`POST`	`/api/team/members`	add somebody with a role
`PUT`	`/api/team/members/:id`	change a name, role, status or password
`DELETE`	`/api/team/members/:id`	remove an account
`GET`	`/api/team/organisations`	organisations with their member/server/credential counts
`POST`	`/api/team/organisations`	super admin: create one
`PUT`	`/api/team/organisations/:id`	rename one
`DELETE`	`/api/team/organisations/:id`	delete an empty one
`POST`	`/api/credentials/test-git`	authenticate a git token *before* saving
`POST`	`/api/credentials/:id/git/account`	re-authenticate and refresh stored account
`POST`	`/api/credentials/:id/git/repositories`	repositories the token can reach

Method	Path	Purpose
`POST`	`/api/credentials/:id/git/branches`	branches of one repository
`POST`	`/api/credentials/:id/git/commits`	recent commits on a branch
`GET`	`/api/servers/status`	just the statuses, for the list to poll — live state laid over the stored rows
`GET`	`/api/servers/:id/history`	every profile collected for a server
`GET`	`/api/git/oauth/providers`	which providers are configured, the callback URL, and the pre-filled registration link
`POST`	`/api/git/oauth/config`	save an OAuth app's client ID/secret to `.env` and use it without restarting
`GET`	`/api/git/oauth/start?kind=`	begin browser sign-in (`github` , `gitlab` or `bitbucket` ; redirects to the provider)
`GET`	`/api/git/oauth/callback`	the provider returns here; token exchanged and account stored
`GET`	`/api/servers`	list servers with a profile summary
`POST`	`/api/servers/test`	test credentials *before* saving
`POST`	`/api/servers`	add a server
`GET`	`/api/servers/:id`	server plus its latest system profile
`PUT`	`/api/servers/:id`	update (blank secret fields keep the stored value)
`DELETE`	`/api/servers/:id`	remove a server and its snapshots
`POST`	`/api/servers/:id/test`	connect and report user, hostname, OS, latency
`POST`	`/api/servers/:id/facts`	collect a fresh system profile
`GET`	`/api/servers/:id/facts`	latest stored profile
`POST`	`/api/servers/:id/exec`	run one command on the server
`GET`	`/api/servers/:id/services`	every systemd service on the host, live
`GET`	`/api/servers/:id/services/:unit`	one unit: properties, unit file, last 200 journal lines
`POST`	`/api/servers/:id/services`	create a systemd unit, then enable and start it
`POST`	`/api/servers/:id/services/:unit/action`	`start` / `stop` / `restart` / `reload` / `enable` / `disable`
`DELETE`	`/api/servers/:id/services/:unit`	remove a unit this panel created
`GET`	`/api/servers/:id/docker`	Docker state, registry sign-ins, networks, volumes and every container
`POST`	`/api/servers/:id/docker/login`	`docker login` on the server, by credential or by hand
`POST`	`/api/servers/:id/docker/logout`	`docker logout` for one registry
`POST`	`/api/servers/:id/docker/networks`	create a user-defined network
`DELETE`	`/api/servers/:id/docker/networks/:name`	remove a network
`DELETE`	`/api/servers/:id/docker/volumes/:name`	remove a named volume and the data in it

Method	Path	Purpose
`GET`	`/api/servers/:id/summary`	what is on this server, counted: containers, domains, jobs, services, runners
`GET`	`/api/servers/:id/cron`	every scheduled job on the machine, from all four places cron keeps them
`POST`	`/api/servers/:id/cron/install`	install cron and start its service
`POST`	`/api/servers/:id/cron/jobs`	add a job to a user's crontab
`PUT`	`/api/servers/:id/cron/jobs`	change one job, matched on its exact line
`DELETE`	`/api/servers/:id/cron/jobs`	remove one job
`GET`	`/api/servers/:id/runners`	the runners on this server as the machine reports them, and any it did not install
`GET`	`/api/servers/:id/stats`	one live sample: CPU, memory, load, disks, network, containers, alerts
`GET`	`/api/servers/:id/stats/stream`	the same, pushed every few seconds over server-sent events
`POST`	`/api/servers/:id/containers/:name/action`	`start` / `stop` / `restart` any container on the host
`GET`	`/api/servers/:id/containers/:name/logs`	that container's recent output
`GET`	`/api/servers/:id/containers/:name/stream`	live stats and live logs for the container page (server-sent events)
`GET`	`/api/servers/:id/containers/:name/compose`	download the container as a `docker-compose.yml`
`GET`	`/api/servers/:id/containers/:name/backup`	download a backup: compose file, inspect and every volume (`?pause=1`)
`GET`	`/api/servers/:id/docker/volumes/:name/download`	download one volume's contents as `.tar.gz`
`GET`	`/api/servers/:id/nginx`	nginx, its sites with their SSL state, and certbot with every certificate
`POST`	`/api/servers/:id/nginx/install`	install `nginx` , or `certbot` with its nginx plugin
`POST`	`/api/servers/:id/nginx/action`	`reload` / `restart` / `start` / `stop` / `test`
`POST`	`/api/servers/:id/nginx/sites`	add a domain — generated from the form, or the file as typed
`GET`	`/api/servers/:id/nginx/sites/:name`	that site's file as it is on the server
`PUT`	`/api/servers/:id/nginx/sites/:name`	edit it, or just enable/disable it
`DELETE`	`/api/servers/:id/nginx/sites/:name`	remove the site; its certificate is kept
`POST`	`/api/servers/:id/nginx/ssl`	get a Let's Encrypt certificate and install it into the vhost
`POST`	`/api/servers/:id/nginx/ssl/renew`	renew one certificate, or everything that is due
`GET`	`/api/installs/catalog`	what can be installed, and the form each one needs
`GET`	`/api/installs/requirements`	`?server_id=&kind=` — what that service needs that the server does not have yet

Method	Path	Purpose
`GET`	`/api/installs`	list installations; `?server_id=` to filter
`POST`	`/api/installs`	install onto the host (Docker, compose, Nginx) or as a container
`POST`	`/api/installs/:id/refresh`	live container state
`POST`	`/api/installs/:id/action`	`start` / `stop` / `restart`
`GET`	`/api/installs/:id/logs`	the container's last lines
`PUT`	`/api/installs/:id/port`	rebuild on a new port / network, keeping the volume
`DELETE`	`/api/installs/:id`	remove the container; `?delete_data=1` drops the volume too
`POST`	`/api/apps/inspect`	is this repository a deployable Node project, and with what settings
`POST`	`/api/apps/ports`	which of these ports are already taken on a server, and by what
`GET`	`/api/apps`	list custom services; `?server_id=` to filter
`POST`	`/api/apps`	start a deployment; answers `202` at once and builds in the background
`POST`	`/api/apps/:id/deploy`	start a rebuild; `409` if one is already running
`POST`	`/api/apps/:id/action`	`start` / `stop` / `restart`
`GET`	`/api/apps/:id/env`	the environment variables as stored (needs the `edit` permission)
`PUT`	`/api/apps/:id/env`	replace them; `apply: true` recreates the containers so they take effect now
`PUT`	`/api/apps/:id/containers`	add, remove or move containers — `ports: [8080, 8081]`, one per container
`GET`	`/api/apps/:id/logs`	one container's recent output; `?container=` picks the instance
`DELETE`	`/api/apps/:id`	remove the container and the clone; `?delete_volumes=1` also deletes its volumes
`GET`	`/api/runners`	list runners; `?credential_id=` or `?server_id=` to filter
`GET`	`/api/runners/:id`	one runner, with its install log
`POST`	`/api/runners`	register with the provider and install on a server
`POST`	`/api/runners/:id/refresh`	live status from GitHub / GitLab
`POST`	`/api/runners/:id/action`	`start` / `stop` / `restart` its systemd service
`DELETE`	`/api/runners/:id`	unregister, remove from the server, forget here
`POST`	`/api/credentials/:id/git/organizations`	organisations (GitHub), groups (GitLab) or workspaces (Bitbucket)
`POST`	`/api/credentials/:id/git/runners`	runners registered for one repository / organisation
`GET`	`/api/credentials`	list (secrets masked); `?provider=mysql` to filter

Method	Path	Purpose
`POST`	`/api/credentials`	add a git / dockerhub / cloudflare / mysql credential
`PUT`	`/api/credentials/:id`	update (blank secret keeps the stored one)
`POST`	`/api/credentials/:id/verify`	verify against the provider
`DELETE`	`/api/credentials/:id`	remove
`POST`	`/api/credentials/test-mysql`	test MySQL details *before* saving
`GET`	`/api/credentials/cloudflare/token-link`	dashboard link with a pre-filled Create API token form
`POST`	`/api/credentials/test-cloudflare`	verify a Cloudflare token and show what it can see, without saving
`POST`	`/api/credentials/cloudflare/connect`	verify a token, read user / accounts / zones, store (re-connecting updates in place)
`POST`	`/api/credentials/:id/cloudflare/account`	re-read and store the account, accounts and zones
`POST`	`/api/credentials/:id/cloudflare/zones/:zoneId`	one domain in full: details, name servers, settings, DNS records
`DELETE`	`/api/credentials/:id/cloudflare/zones/:zoneId`	remove the domain from Cloudflare; body `{ "confirm": "<domain name>" }`
`POST`	`/api/credentials/:id/cloudflare/zones/:zoneId/dns`	DNS records of one zone
`POST`	`/api/credentials/:id/cloudflare/zones/:zoneId/dns/records`	add a DNS record (A, AAAA, CNAME, TXT, MX, NS, PTR); name may be `@`, `www` or fully qualified
`PUT`	`/api/credentials/:id/cloudflare/zones/:zoneId/dns/records/:recordId`	change a DNS record
`DELETE`	`/api/credentials/:id/cloudflare/zones/:zoneId/dns/records/:recordId`	delete a DNS record
`POST`	`/api/credentials/:id/mysql/overview`	engine state, databases, users, biggest tables
`POST`	`/api/credentials/:id/mysql/databases/:db`	tables, views and routines in one database
`POST`	`/api/credentials/:id/mysql/databases/:db/tables/:table`	columns, indexes, foreign keys
`POST`	`/api/credentials/:id/mysql/query`	read-only query
`POST`	`/api/credentials/:id/mysql/charsets`	character sets and collations
`POST`	`/api/credentials/:id/mysql/schemas`	create a database `{ name, charset, collation }`
`PUT`	`/api/credentials/:id/mysql/databases/:db`	change a database's charset / collation
`DELETE`	`/api/credentials/:id/mysql/databases/:db`	drop a database `{ confirm: "<db name>" }`
`POST`	`/api/credentials/:id/mysql/users/list`	accounts, state, sessions and grants
`POST`	`/api/credentials/:id/mysql/users`	create a user `{ user, host, password, database?, privileges? }`
`PUT`	`/api/credentials/:id/mysql/users`	password, lock / unlock, max connections `{ user, host, … }`

Method	Path	Purpose
`DELETE`	`/api/credentials/:id/mysql/users`	drop a user `{ user, host }`
`POST`	`/api/credentials/:id/mysql/users/grants`	grant `{ user, host, database, privileges, grantOption }`
`DELETE`	`/api/credentials/:id/mysql/users/grants`	revoke everything on one database `{ user, host, database }`
`POST`	`/api/credentials/:id/mysql/variables`	editable settings and every global variable
`PUT`	`/api/credentials/:id/mysql/variables`	`SET GLOBAL` / `SET PERSIST` `{ name, value, persist }`
`POST`	`/api/credentials/test-db`	try PostgreSQL / MongoDB / Redis / Elasticsearch details before saving
`POST`	`/api/credentials/:id/db/overview` · `/db/databases` · `/db/database` · `/db/item` · `/db/users/list` · `/db/config` · `/db/query`	PostgreSQL / MongoDB / Redis / Elasticsearch reads (viewers allowed)
`POST` / `DELETE`	`/api/credentials/:id/db/schemas` · `/db/databases`	create / drop (flush) a database
`POST` / `PUT` / `DELETE`	`/api/credentials/:id/db/users`	create / change / drop a user, identified by `key`
`POST` / `DELETE`	`/api/credentials/:id/db/users/grants`	grant / revoke
`PUT`	`/api/credentials/:id/db/config`	change a setting `{ name, value, persist }`
`GET`	`/api/activity`	last 50 actions

Layout

```
src/
  index.js            Express app, startup, route wiring
  config.js           env, paths, master key
  db/index.js         MySQL pool, schema, org migration, query helpers, activity log
  db/importSqlite.js  one-time import from the old SQLite store
  lib/crypto.js       AES-256-GCM encrypt / decrypt / mask
  lib/auth.js         scrypt passwords, session tokens, cookies, the role table
  lib/authGuard.js    who is signed in, and what their role may do
  lib/context.js      the current user and organisation, for code far from the route
  lib/git.js          GitHub / GitLab / Bitbucket accounts, repositories, branches, commits, runners
  lib/gitAccounts.js  loading a git credential and keeping its token fresh
  lib/oauth.js        browser sign-in: state, PKCE, code exchange, token refresh
  lib/envFile.js      updates .env in place, keeping comments and ordering
  routes/gitOauth.js  the /start and /callback endpoints and their result page
  lib/ssh.js          connection handling, exec, running a script as root, friendly error mapping
  lib/services.js     systemd: list, describe, control, create and delete units
  lib/runnerInstall.js  the remote install / removal scripts for GitHub and GitLab runners
  lib/catalog.js      what can be installed and the form each entry needs
  lib/nodeApp.js      reads a repository's package.json and writes its Dockerfile
  lib/deploy.js       the remote clone / build / push / run script
  lib/docker.js       Docker over SSH: state, networks, containers, engine install
  lib/probe.sh        the remote Ubuntu collection script
```

```
  lib/systemInfo.js   parses probe output into structured facts
  lib/mysql.js        MySQL connections (direct or SSH-tunnelled), inspection, read-only queries,
database / user / grant / setting management
  routes/servers.js   server CRUD, connect, collect, systemd services
  routes/credentials.js  provider credentials, verification, MySQL browsing
  routes/runners.js   CI runners: register, install, control, remove
  routes/installs.js  the catalog, installing, port changes, removal
  routes/apps.js      custom services: inspect, deploy, redeploy, logs
  routes/auth.js      setup, sign in, sign out, password, organisation switch
  routes/team.js      members and organisations
public/               single-page UI (no build step)
data/                 master key (and the archived SQLite file) — gitignored
```

All panel data — servers, their collected system profiles (as a JSON column), credentials, projects and the activity log — lives in your MySQL database. Only the encryption key stays on disk.

What is next

The `projects` table is already in place, linking a server to a git, Docker Hub and Cloudflare credential plus a repo, branch and domain. The deployment pipeline and Cloudflare DNS management build on top of it. "# aj-pilot" "# aj-pilot"